Formation Data Engineer — RNCP 40167

TradeCorp Data Platform

Jalon 2, Partie 2 — Programmation modulaire

JALON 2, PARTIE 2 — Programmation modulaire

Contexte

Dans la Partie 1, vous avez exploré, nettoyé, joint et agrégé les données TradeCorp dans une succession de notebooks Jupyter. Vous avez sans doute remarqué une limite au fil du travail : chaque notebook redémarre avec une SparkSession vide, et pour éviter de tout réexécuter à chaque fois, vous avez dû sauvegarder des résultats intermédiaires en Parquet entre deux notebooks (Q20 bis). C'est exactement le symptôme d'un code qui n'est pas encore réutilisable : les mêmes transformations se retrouvent copiées-collées d'un notebook à l'autre, et rien de ce que vous avez écrit ne peut tourner seul, sans navigateur, sans intervention manuelle.

Le DSI de TradeCorp vous demande maintenant de transformer ce prototype en un vrai code de production : des fichiers Python modulaires, chacun avec une seule responsabilité, testés, versionnés, et capables de s'exécuter depuis un simple terminal — sans notebook ouvert, sans copier-coller. C'est cette version industrialisée, et uniquement elle, qui sera orchestrée au Jalon 3 : Airflow ne fera qu'appeler vos scripts dans le bon ordre et au bon moment — vous n'y écrirez aucune nouvelle logique de transformation, seulement de l'orchestration.

Vous quittez donc JupyterLab pour VS Code (attaché au conteneur Spark via Dev Containers), et vous construisez, étape par étape, l'architecture suivante :

```
tradecorp/
├── src/
│   ├── utils.py                    → fonctions nettoyage et connection ADLS
│   ├── reader.py                   → lecture ADLS (CSV + fichiers de référence)
│   ├── transformer.py              → jointures et DataFrame enrichi (Q24a/Q24b)
│   ├── enrichment.py               → conversion devise (taux + mapping pays)
│   ├── writer.py                   → écriture Parquet + upload ADLS
│   ├── pipeline.py                 → orchestration complète, logging, gestion
d'erreurs
│   ├── fetch_exchange_rates.py     → appel API, upload du taux du jour
│   └── upload_country_currency.py  → upload unique du mapping pays/devise
└── tests/
    ├── test_transformers.py        → tests pytest
    └── run_tests.py                → lance pytest via spark-submit
```

Chaque fichier a une seule responsabilité : reader lit, transformer transforme, writer écrit, pipeline orchestre. C'est cette séparation stricte qui rendra le branchement Airflow trivial au jalon suivant une tâche Airflow, un script, une commande.

Ce que doit accomplir chaque fichier

> 💡 *La façon de découper le code en fonctions vous appartient — c'est aussi ça, la programmation modulaire : trouver le bon découpage. Ci-dessous, uniquement ce que chaque fichier doit accomplir, pas comment l'organiser en interne.*

src/utils.py

- Des fonctions qui permettent la connexion à ADLS Gen2 (se connecter au compte de stockage, télécharger un fichier depuis un conteneur de données)
- Des fonctions de transformation, une par table source, qui appliquent ce qui a été fait en Jalon 2 Partie 1 : suppression des espaces inutiles (trim), harmonisation de la casse (majuscules/minuscules selon la colonne), retypage des colonnes concernées (dates en DateType, prix en DoubleType, quantités en IntegerType), gestion des valeurs manquantes, calcul du sous-total, ajout des colonnes calculées (is_shipped, en_stock, full_name)

src/reader.py

- Télécharge les 8 fichiers CSV métier depuis la zone raw d'ADLS Gen2 et les lit avec Spark
- Télécharge et lit également les deux fichiers de référence (country_currency.csv et exchange_rates.json) utilisés pour l'enrichissement

src/transformer.py

- Applique les fonctions de nettoyage de utils.py à chaque table
- Joint les 6 tables pour construire le DataFrame enrichi, en gérant les colonnes en doublon comme vu en Q24a/Q24b

src/enrichment.py

- Détermine la devise du pays de chaque client (à partir du fichier de référence)
- Applique le taux de change du jour à sous_total pour produire un montant dans la devise locale du client (sous_total_local)

src/writer.py

- Réécrit les données transformées (et enrichies) en Parquet, puis les envoie vers la zone clean d'ADLS Gen2

src/pipeline.py

- Enchaîne dans l'ordre : lecture → transformation → enrichissement → écriture
- Journalise chaque étape (logging, pas print) et gère les erreurs proprement (la SparkSession doit s'arrêter même en cas d'échec)

src/fetch_exchange_rates.py

- Appelle l'API de taux de change et dépose le résultat dans ADLS Gen2 (zone raw, sous-dossier reference/) => script indépendant, sans dépendance Spark

src/upload_country_currency.py

- Dépose le fichier country_currency.csv fourni dans ADLS Gen2 (zone raw, sous-dossier reference/) — à exécuter une seule fois, pas à chaque run du pipeline

5.1 — Les modules Python

Q38 — Créer reader.py

Créer src/reader.py : télécharger les 8 fichiers CSV métier depuis le conteneur raw d'ADLS Gen2 vers un dossier temporaire du conteneur, puis les lire avec Spark. À vous de choisir comment découper cela en fonctions. Tester en l'exécutant directement : docker exec tradecorp_spark spark-submit /home/jovyan/src/reader.py

Indice : utiliser le SDK azure-storage-blob (BlobServiceClient) pour le téléchargement, puis spark.read.csv(path, header=True, inferSchema=True) pour la lecture. Ne télécharger que les 8 fichiers CSV métier nommés explicitement ; pas tout le conteneur sans filtre

Q39 — Créer transformer.py

Créer src/transformer.py qui applique les fonctions de nettoyage de utils.py pour construire le DataFrame enrichi final.

Liste des transformations attendues du fichier utils.py :

- clean_customers(df) : nettoie la table des clients
  - company_name : supprime les espaces avant/après (trim)
  - contact_name : supprime les espaces, puis met en forme première lettre en majuscule (jean dupont " → "Jean Dupont")
  - country : supprime les espaces, puis met tout en majuscules ("france" → "FRANCE")
  - Supprime les lignes en double sur customer_id (garde un seul exemplaire par client)

- clean_orders(df) — nettoie la table des commandes
  _ Supprime les commandes qui n'ont pas de date de livraison (shipped_date vide) (ici on considère qu'une commande non livrée n'est pas exploitable pour l'analyse)
  _ Convertit trois colonnes de type texte en vraies dates : order_date, required_date, shipped_date
  _ Convertit freight (frais de port) en nombre décimal
  _ Renomme la colonne ship_via en shipper_id (nom plus clair)
  _ Ajoute une nouvelle colonne is_shipped : True si la commande a bien une date de livraison, False sinon

- clean_order_details(df) — nettoie le détail des lignes de commande
  _ Convertit unit_price en nombre décimal, quantity en nombre entier
  _ Convertit discount (remise) en nombre décimal
  _ Renomme unit_price en prix_unitaire, quantity en quantite (mise en français, cohérente avec le reste du projet)

- add_sous_total(df) — calcule le montant d'une ligne de commande
  _ Ajoute une colonne sous_total = prix unitaire × quantité × (1 - remise), arrondie à 2 décimales
  _ Exemple : 10€ × 2 unités avec 10% de remise = 10 × 2 × 0.9 = 18.0

- clean_employees(df) — nettoie la table des employés
  _ Ne garde que les colonnes utiles : employee_id, first_name, last_name, title, hire_date, city, country (retire les colonnes inutiles comme la photo ou les notes)
  _ Ajoute une colonne full_name = prénom + espace + nom (ex. "Jean" + "Dupont" → "Jean Dupont")

- clean_products(df) — nettoie la table des produits
  _ Convertit unit_price en nombre décimal
  _ Ajoute une colonne en_stock : True si le stock (units_in_stock) est supérieur à 0, False sinon

Transformation attendu dans le fichier trasformer.py :

- build_enriched(dataframes) => construit la table finale enrichie

Ce que fait la fonction :

- Nettoie individuellement les tables clients, commandes, lignes de commande, produits et employés (appel aux fonctions de utils.py)
- Utilise catégories et transporteurs tels quels (pas de nettoyage nécessaire pour ces deux tables)
- Ajoute le nom de la catégorie à chaque produit
- Joint les 7 tables ensemble (lignes de commande, commandes, clients, produits, catégories, employés, transporteurs) pour réunir toutes les informations sur une seule ligne par produit commandé
- Sélectionne uniquement les colonnes utiles, en renommant celles qui existent dans plusieurs tables (comme country ou company_name) pour éviter toute confusion entre le pays du client et celui du transporteur

Le schema attendu est le suivant pour la partie transform :

```
root
 |-- order_id: integer (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- employee_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- required_date: date (nullable = true)
 |-- shipped_date: date (nullable = true)
 |-- freight: double (nullable = true)
 |-- is_shipped: boolean (nullable = true)
 |-- prix_unitaire: double (nullable = true)
 |-- quantite: integer (nullable = true)
 |-- discount: double (nullable = true)
 |-- sous_total: double (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- customer_country: string (nullable = true)
 |-- customer_city: string (nullable = true)
 |-- product_name: string (nullable = true)
 |-- category_name: string (nullable = true)
 |-- en_stock: boolean (nullable = true)
 |-- full_name: string (nullable = true)
 |-- shipper_name: string (nullable = true)
```

order_id	customer_id	employee_id	product_id	order_date	required_date	shipped_date	freight	is_shipped	prix_unitaire	quantite	discount	sous_total	customer_name	customer_country	customer_city	product_name	category_name	en_stock	full_name	shipper_name
10341	SIMOB	7	33	1996-10-29	1996-11-26	1996-11-05	26.78	true	2.0	8	0.0	16.0	Simons bistro	DENMARK	Kobenhavn	Geitost	Dairy Products	true	Robert King	Federal Shipping
10341	SIMOB	7	59	1996-10-29	1996-11-26	1996-11-05	26.78	true	44.0	9	0.15	336.6	Simons bistro	DENMARK	Kobenhavn	Raclette Courdavault	Dairy Products	true	Robert King	Federal Shipping
10367	VAFFE	7	34	1996-11-28	1996-12-26	1996-12-02	13.55	true	11.2	36	0.0	403.2	Vaffeljernet	DENMARK	Århus	Sasquatch Ale	Beverages	true	Robert King	Federal Shipping
10367	VAFFE	7	54	1996-11-28	1996-12-26	1996-12-02	13.55	true	5.9	18	0.0	106.2	Vaffeljernet	DENMARK	Århus	Tourtière	Meat/Poultry	true	Robert King	Federal Shipping
10367	VAFFE	7	65	1996-11-28	1996-12-26	1996-12-02	13.55	true	16.8	15	0.0	252.0	Vaffeljernet	DENMARK	Århus	Louisiana Fiery H...	Condiments	true	Robert King	Federal Shipping

Q40 — Créer writer.py

Créer src/writer.py qui écrit le DataFrame en Parquet dans le conteneur clean d'ADLS Gen2.

Q41 — Créer pipeline.py

Créer src/pipeline.py qui importe reader, transformer et writer, et orchestre le pipeline de bout en bout : lecture → transformation → écriture.

Q42 — Lancer avec spark-submit

Exécuter le pipeline depuis le terminal du conteneur : docker exec tradecorp_spark spark-submit /home/jovyan/src/pipeline.py. Vérifier que tout tourne sans erreur, du téléchargement ADLS jusqu'à l'upload du Parquet final.

5.2 — Tests unitaires avec pytest

Q43 — Installer pytest

Ajouter pytest dans requirements.txt, puis reconstruire l'image : docker compose up -d --build.

> ⚠ *Ne pas installer pytest à la volée avec docker exec tradecorp_spark pip install pytest — cette installation serait perdue au prochain redémarrage du conteneur. L'ajouter dans requirements.txt la rend permanente.*

Q44 — Créer test_transformers.py

Créer tests/test_transformers.py. Écrire un test qui vérifie que clean_orders() supprime bien les lignes où shipped_date est null.

Q45 — Test sur sous_total

Écrire un test qui vérifie que add_sous_total() calcule correctement le sous-total. Créer un petit DataFrame de test avec des valeurs connues et vérifier le résultat.

Q46 — Test sur clean_customers

Écrire un test qui vérifie que clean_customers() applique bien la suppression des espaves et la majuscule à la première lettre sur contact_name, et le nom du pays en majuscule.

Q47 — Créer run_tests.py

Créer tests/run_tests.py, un script qui exécute pytest via pytest.main(["/home/jovyan/tests/test_transformers.py", "-v"]).

> ⚠ *Cette étape est indispensable : lancer pytest directement (sans passer par Spark) provoque une erreur JAVA_GATEWAY_EXITED, car pyspark a besoin de la JVM initialisée par spark-submit pour fonctionner correctement dans ce conteneur.*

Q48 — Lancer les tests

Lancer tous les tests depuis le conteneur : docker exec tradecorp_spark spark-submit /home/jovyan/tests/run_tests.py. Tous les tests doivent passer.

5.3 — Enrichissement : devise et taux de change (C33)

> 💡 *Deux données de nature différente : le mapping pays→devise est stable (fichier de référence écrit une fois), le taux de change varie chaque jour (récupéré via API à chaque exécution). Les deux sont stockés dans ADLS Gen2, conteneur raw, sous-dossier reference/.*

Q49 — Créer et uploader country_currency.csv

Uploader dans ADLS le fichier country_currency.csv dans le repertoire : raw/reference/country_currency.csv

Q50 — Créer fetch_exchange_rates.py

Créer src/fetch_exchange_rates.py, un script Python sans dépendance Spark, qui appelle l'API gratuite exchangerate-api.com (voir lien ci-dessous) et uploade la réponse JSON brute vers raw/reference/exchange_rates.json.

🔗 **API à utiliser :** https://api.exchangerate-api.com/v4/latest/USD

> 💡 *Gratuite, sans clé, sans compte à créer. Renvoie un JSON avec base (USD), date, et rates (dictionnaire de ~166 codes devises ISO). Ne pas confondre avec api.restcountries.com ou d'autres services similaires qui nécessitent une inscription et/ou une clé payante.*

Q51 — Compléter reader.py

Ajouter à reader.py la possibilité de télécharger et lire les deux fichiers de référence (country_currency.csv et exchange_rates.json).

Q52 — Créer enrichment.py

Créer src/enrichment.py : déterminer la devise du pays de chaque client à partir du fichier de référence, puis appliquer le taux de change du jour à sous_total pour produire sous_total_local.

Q53 — Compléter pipeline.py

Brancher l'enrichissement dans pipeline.py, entre build_enriched() et l'écriture : télécharger les deux fichiers de référence, appeler add_currency_column(). Ajouter un logging structuré (module logging, horodaté, plutôt que print) et une gestion d'erreurs (try/except/finally garantissant spark.stop() même en cas d'échec).

Q54 — Test de l'enrichissement

Ajouter test_add_currency_column dans test_transformers.py, avec un taux simulé (dict Python codé en dur dans le test), sans dépendre d'un appel réseau réel.

Q55 — Relancer et versionner

Relancer spark-submit /home/jovyan/tests/run_tests.py, mes 4 tests doivent passer. Relancer spark-submit /home/jovyan/src/pipeline.py de bout en bout et vérifier que currency et sous_total_local apparaissent dans le Parquet final. Commiter et pousser tout le code sur GitHub.

```
root
 |-- order_id: integer (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- employee_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- required_date: date (nullable = true)
 |-- shipped_date: date (nullable = true)
 |-- freight: double (nullable = true)
 |-- is_shipped: boolean (nullable = true)
 |-- prix_unitaire: double (nullable = true)
 |-- quantite: integer (nullable = true)
 |-- discount: double (nullable = true)
 |-- sous_total: double (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- customer_country: string (nullable = true)
 |-- customer_city: string (nullable = true)
 |-- product_name: string (nullable = true)
 |-- category_name: string (nullable = true)
 |-- en_stock: boolean (nullable = true)
 |-- full_name: string (nullable = true)
 |-- shipper_name: string (nullable = true)
 |-- currency: string (nullable = true)
 |-- sous_total_local: double (nullable = true)
```